

DATA-STRUCTURES & ALGORITHMS

with

Manoj Prabhakaran

Lecture 5

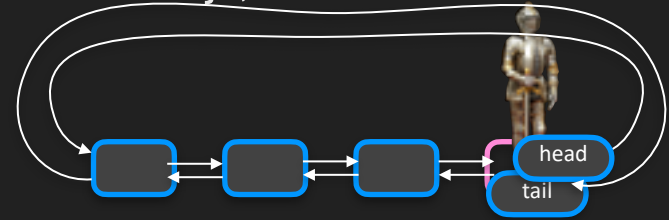
Linked Lists: Some Cool Tricks!

Recap: Doubly-Linked List w/ Sentinel Node

- ```
struct node { int val; node *next, *prev; };
struct list { node sentinel; /*functions*/};
```

- Insert/Erase code is simplified, and without if-else conditions

- Per list, one "val" field in the sentinel is wasted
- A significant issue only if a large number of *small* lists are being used



# Recap: Doubly-Linked List w/ Sentinel Node

```
struct node { int val; node *next, *prev; };
struct list {
 node sentinel; // sentinel.next is head, sentinel.prev is tail
 list() { sentinel.next = sentinel.prev = &sentinel; } // empty list. no nullptr
 node* begin() { return sentinel.next; } // begin points to head
 node* end() { return &sentinel; } // end points beyond tail, to the sentinel itself
 void insert(int x, node* p) { // insert before. for insert_after can use insert(x,p->next)
 node* tmp = new node{x,p,p->prev};
 p->prev->next = tmp; // no special case for p being head
 p->prev = tmp;
 }
 void erase(node* p) { // assumes p a non-nullptr
 if(p==end()) return; // sentinel cannot be removed
 p->prev->next = p->next;
 p->next->prev = p->prev;
 delete p;
 }
 // Exercise: ~list(), find(), size(), reverse(), join(), ... Add members as needed.
};
```



# In std::list

- Very similar to our doubly-linked list with a sentinel. Needn't be circular.
- Doesn't expose a separate node struct. **Instead of node\*** uses a type called **std::list::iterator**
  - Returned by begin() and end()
  - Supports \*, ->, ++, --, ==, !=
    - ⚠ Effects of --L.begin(), ++L.end() and \*(L.end()) undefined
  - **std::find**(L.begin(), L.end(), x) returns an iterator to where the search ended — a node in the list with value x, or otherwise L.end()

\*it instead of p->val.  
it->field instead of  
p->val.field

++it and --it  
instead of  
p=p->next,  
p=p->prev

# Some Clever Algorithms

- Today
  - An algorithm for efficiently finding a cycle in a (malformed) linked list
  - An algorithm for detecting intersection of two linked lists
  - Lists reversible in  $O(1)$  time
- First:
  - Merging two sorted linked lists (a useful subroutine in the mergesort algorithm)

# Merging Sorted Lists

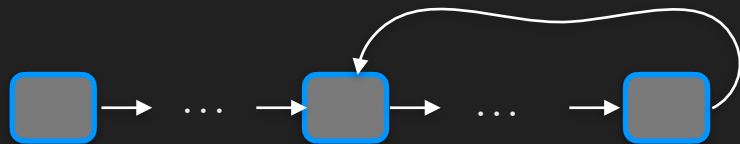
- Problem: Given two sorted (singly) linked lists, L1, L2, merge them into a single sorted list, using  $O(1)$  additional space, and without moving any nodes (so that pointers to nodes remain valid after merging)
- Idea: “detach” a node from the front of one of the two lists (whichever has the smaller value) and “attach” it to the back of a merged list

```
struct node { int val; node* next = nullptr; };
struct list {
 node* head=nullptr; node* tail = nullptr;
 bool empty() { return (head==nullptr); }
 node* detach_front(); // doesn't delete detached node
 void attach_back(node* p); // doesn't make a new node
};
```

 Exercise

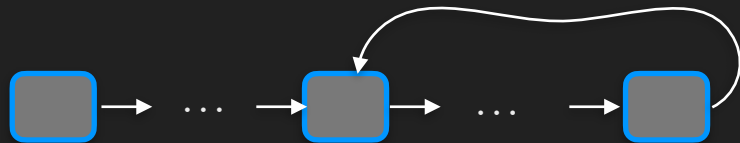
# Cycle Detection

- Recall a singly-linked list
- What if, when creating the list, by mistake, a node points to the head?
  - This is a (singly-linked) circular linked list
  - How can you check if this has happened?
    - Easy: Traverse the list until we hit a nullptr or the head again
- A trickier situation: what if a node points to some previous node (not necessarily the head)?
  - Don't know which node will repeat



# Cycle Detection

- With  $\Theta(n)$  extra space, can solve it easily
  - If each node has space that can be used to “mark” it, then just mark all nodes seen during a traversal. Cycle iff we see a previously marked node.
  - Create a hash table (or set) of all the node addresses encountered, looking for duplicates while adding.
- With just  $O(1)$  extra space?
  - In  $\Theta(n^2)$  time: At each node, check if its next node is reached first, when starting from the head, through another node: Extra time =  $1 + 2 + \dots + n$
- In  $O(1)$  extra space and  $O(n)$  time?





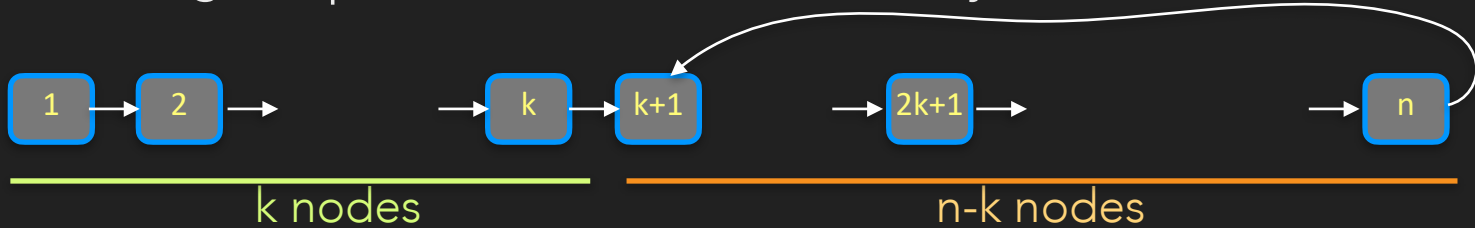
# Floyd's Cycle Detection

- ```
node* slow = head; node* fast = head;
while(fast && fast->next) {
    slow=slow->next; fast=fast->next->next;
    if(slow == fast) return true; // cycle!
}
return false; // no cycle
```

 - In a straight list: `slow` will never catch up with `fast`, before `fast` (or `fast->next`) reaches the end of the list and we will return `false`
 - If there is a cycle: at some point both `slow` and `fast` will be in the cycle (because once you enter, you never leave). After that `fast` is chasing and catching up with `slow`, reducing the gap by one at each step. The two will meet!

Floyd's Cycle Detection

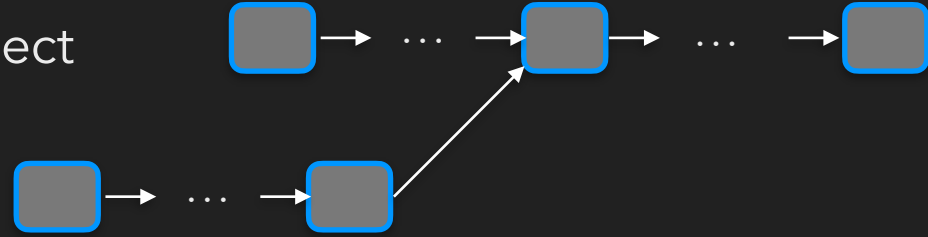
- We can even get a pointer to the start of the cycle!



- For simplicity, let us pretend $n \gg k$.
- In k steps: slow reaches node $k+1$, fast reaches node $2k+1$ (within the cycle)
- In the cycle, slow is $(n-2k)$ steps ahead of fast (slow is at "position $n+1$ ")
 - In $n-2k$ more steps, they meet at position $n-k+1$ (or $2(n-k)+1$)
 - Now move one pointer to head, and move both forward in sync till they meet: In k more steps both reach position $k+1$

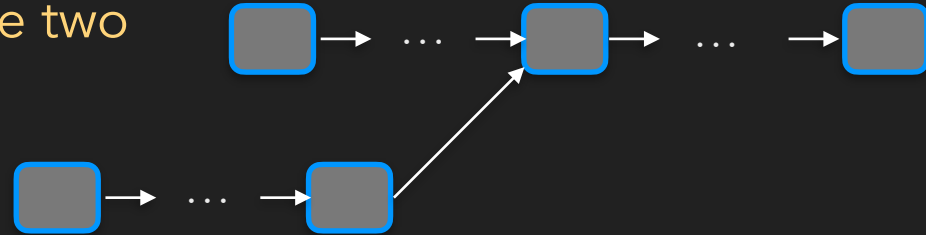
Intersection Detection

- Suppose two (singly-linked) lists intersect
- To find the first node that belongs to both lists
- Easy using $O(n)$ extra space: Store all the addresses of the nodes of one list in a data structure (set or hash table), and go through each element of the second list checking if it is stored already
- Easy using $O(n^2)$ time and $O(1)$ extra space: For each node in the second list, scan the entire first list to see if it is present there
- Can we do it in $O(n)$ time with $O(1)$ extra space?



Intersection Detection

- Suppose we were guaranteed that the two lists have the same length
- Then, there is some index i such that both lists have their i^{th} node identical
 - Traverse the two lists in synch, checking if both are at the same node
- What if we're not guaranteed same length?
- We can find the lengths of both the lists in $O(n)$ time
- If the lengths are not the same, we can skip the first nodes in the longer list, so that it is left with exactly as many nodes as the shorter one. Now as above.



O(1) Reversal

Challenge: Design it so that it supports O(1) reversal of segments
Hint: For each node, allow either pointer to be next; no need to store this bit, but it can be inferred at traversal time (how?)

- Consider designing a doubly-linked list
 - Suppose we want to design it so that it can be quickly reversed

- Idea: store a bit in the list indicating which is the forward direction:

```
struct node { int val; node *A, *B; };  
struct list { node S; bool A_is_next; /*...*/};  
// if A_is_next, then A is head, else S.B is head
```

- To reverse the list, simply toggle this bit:

```
void list::reverse() { A_is_next = !A_is_next; }
```

- In other functions, instead of p->next, p->prev, use **next(p)**, **prev(p)**:

```
node*& list::next(node* p) { return A_is_next? p->A : p->B; }  
node*& list::prev(node* p) { return A_is_next? p->B : p->A; }
```

How will you
redefine O(1)
insert?